SSH Certificate Authority (CA) — Complete Tutorial

Table of Contents

1. Why SSH Certificates?
2. Concepts
3. Part 1: Local CA (Learning Setup)
4. Part 2: Server-Side Configuration
5. Part 3: Signing User Keys
6. Part 4: Host Certificates (Servers Prove Identity)
7. Part 5: Principals and Access Control
8. Part 6: Production CA Server with HashiCorp Vault
9. Part 7: Revocation
10. Part 8: Automation and Tooling
11. Architecture Diagrams
12. Quick Reference
13. Troubleshooting

---

1. Why SSH Certificates?

The problem with traditional SSH keys

With plain SSH keys, every user's public key must be added to `~/.ssh/authorized_keys` on every server they need access to. This creates:

- **Key sprawl** — hundreds of keys across dozens of servers, no central inventory
- **No expiration** — keys live forever unless manually removed
- **No identity** — a key on a server tells you nothing about who it belongs to
- **Painful offboarding** — when someone leaves, you hunt across every server to remove their key
- **No audit trail** — no record of when/why a key was granted access

What certificates solve

With an SSH CA:

- Servers trust **one CA public key**, not individual user keys
- Certificates **expire automatically** (you set the TTL)
- Certificates carry **identity** (who), **principals** (what they can do), and **constraints**
- Offboarding = revoke in IdP or CA policy; short-lived certs expire on their own
- Every certificate issuance is a **logged, auditable event**

---

2. Concepts

Terminology

Term	Definition
CA (Certificate Authority)	A key pair whose private key signs other keys. The public key is distributed to servers.
Certificate	A public key + metadata (identity, principals, expiry) signed by the CA.

Principal	A username or role that a certificate is authorized to use. Maps to system users on the server.
TTL (Time to Live)	How long a certificate is valid.
KRL (Key Revocation List)	A list of revoked certificates that servers check before granting access.
HSM (Hardware Security Module)	A dedicated device that stores the CA private key and performs signing operations.

Types of SSH certificates

1. **User certificates** — prove a user's identity to a server
2. **Host certificates** — prove a server's identity to a user (eliminates "trust this fingerprint?" prompts)

---

3. Part 1: Local CA (Learning Setup)

This section sets up everything locally so you can experiment before moving to production.

3.1 Create a working directory

```
mkdir -p ~/ssh-ca-lab/{ca,users,servers}
cd ~/ssh-ca-lab
```

3.2 Generate the CA key pair

Use Ed25519 — it's the strongest and fastest algorithm available in OpenSSH.

```
# Generate the CA key with a strong passphrase
ssh-keygen -t ed25519 -f ca/ca_key -C "My SSH CA"
```

You now have:

```
ca/ca_key       # PRIVATE — guards everything. Protect with your life.
ca/ca_key.pub   # PUBLIC  — distribute to servers. Safe to share.
```

> ***Security note**: In production, `ca_key` should NEVER exist as a plain file on any networked machine. Use an HSM, Vault, or KMS. For learning, a local file is fine.*

3.3 Inspect the CA public key

```
cat ca/ca_key.pub
```

Output looks like:

```
ssh-ed25519 AAAAC3Nza... My SSH CA
```

This single line is all that servers need to trust your CA.

---

4. Part 2: Server-Side Configuration

4.1 Copy the CA public key to the server

```
scp ca/ca_key.pub user@server:/tmp/ca_key.pub
```

Then on the server:

```
sudo mv /tmp/ca_key.pub /etc/ssh/ca_key.pub
sudo chmod 644 /etc/ssh/ca_key.pub
sudo chown root:root /etc/ssh/ca_key.pub
```

4.2 Configure sshd to trust the CA

Edit `/etc/ssh/sshd_config` on the server:

```
# Trust certificates signed by this CA for user authentication
TrustedUserCAKeys /etc/ssh/ca_key.pub
```

Restart sshd:

```
sudo systemctl restart sshd
```

> ***What this does**: sshd will now accept any user certificate signed by `ca_key`, as long as the certificate's principals match (covered in Part 5).*

4.3 Verify the configuration

```
sudo sshd -T | grep trustedusercakeys
```

Should output:

```
trustedusercakeys /etc/ssh/ca_key.pub
```

---

5. Part 3: Signing User Keys

5.1 Generate a user key pair

This is the user's own key — they generate it themselves.

```
ssh-keygen -t ed25519 -f users/alice -C "alice@company.com"
```

Files created:

```
users/alice       # User's private key (stays with user)
users/alice.pub   # User's public key (sent to CA for signing)
```

5.2 Sign the user's public key with the CA

The CA operator does this:

```
ssh-keygen -s ca/ca_key \
  -I "alice@company.com" \
  -n alice,deploy \
  -V +8h \
  users/alice.pub
```

Flags explained:

Flag	Value	Purpose
`-s ca/ca_key`	CA private key	Signs with this CA
`-I "alice@company.com"`	Key identity	Shows up in logs — use email or employee ID
`-n alice,deploy`	Principals	Comma-separated list of usernames this cert can log in as
`-V +8h`	Validity	Certificate expires in 8 hours

This creates:

```
users/alice-cert.pub   # The signed certificate
```

5.3 Inspect the certificate

```
ssh-keygen -L -f users/alice-cert.pub
```

Output:

```
users/alice-cert.pub:
        Type: ssh-ed25519-cert-v01@openssh.com user certificate
        Public key: ED25519-CERT SHA256:xxxx
        Signing CA: ED25519 SHA256:yyyy (using ssh-ed25519)
        Key ID: "alice@company.com"
        Serial: 0
        Valid: from 2026-03-09T10:00:00 to 2026-03-09T18:00:00
        Principals:
                alice
                deploy
        Critical Options: (none)
        Extensions:
                permit-X11-forwarding
                permit-agent-forwarding
                permit-port-forwarding
                permit-pty
                permit-user-rc
```

5.4 Use the certificate to connect

The user places the cert alongside their key:

```
# Ensure both files are in ~/.ssh/
cp users/alice ~/.ssh/id_ed25519_alice
cp users/alice-cert.pub ~/.ssh/id_ed25519_alice-cert.pub

# SSH automatically uses the cert
ssh -i ~/.ssh/id_ed25519_alice alice@server
```

> ***How SSH finds the cert**: when you specify `-i keyfile`, SSH automatically looks for `keyfile-cert.pub` in the same directory.*

5.5 Certificate options and restrictions

You can restrict what a certificate holder can do:

```
# No port forwarding, no agent forwarding, no X11 — only a PTY
ssh-keygen -s ca/ca_key \
  -I "alice@company.com" \
  -n alice \
  -V +8h \
  -O no-port-forwarding \
  -O no-agent-forwarding \
  -O no-x11-forwarding \
  users/alice.pub
```

Lock to a source IP range:

```
ssh-keygen -s ca/ca_key \
  -I "alice@company.com" \
  -n alice \
  -V +8h \
  -O source-address=10.0.0.0/8 \
  users/alice.pub
```

Force a specific command (e.g., for CI/CD):

```
ssh-keygen -s ca/ca_key \
  -I "ci-deploy-bot" \
  -n deploy \
  -V +1h \
  -O force-command="/usr/local/bin/deploy.sh" \
  users/ci_bot.pub
```

---

6. Part 4: Host Certificates (Servers Prove Identity)

This eliminates the "The authenticity of host ... can't be established. Are you sure?" prompt.

6.1 Sign the server's host key

On the CA machine:

```
# Copy the server's host public key
scp root@server:/etc/ssh/ssh_host_ed25519_key.pub servers/server1.pub

# Sign it as a HOST certificate (-h flag)
ssh-keygen -s ca/ca_key \
  -I "server1.example.com" \
  -h \
  -n server1.example.com,10.0.1.50 \
  -V +52w \
  servers/server1.pub
```

Flag	Purpose
`-h`	This is a HOST certificate, not a user certificate
`-n server1.example.com,10.0.1.50`	Hostnames/IPs the cert is valid for
`-V +52w`	Valid for 1 year

6.2 Install the host certificate on the server

```
scp servers/server1-cert.pub root@server:/etc/ssh/ssh_host_ed25519_key-cert.pub
```

On the server, add to `/etc/ssh/sshd_config`:

```
HostCertificate /etc/ssh/ssh_host_ed25519_key-cert.pub
```

Restart sshd:

```
sudo systemctl restart sshd
```

6.3 Configure clients to trust the CA for host verification

On each developer's machine, add to `~/.ssh/known_hosts`:

```
@cert-authority *.example.com ssh-ed25519 AAAAC3Nza...<contents of ca_key.pub>
```

Or system-wide in `/etc/ssh/ssh_known_hosts`.

Now connecting to any `*.example.com` host with a valid host certificate will be automatically trusted — no fingerprint prompts.

---

7. Part 5: Principals and Access Control

This is the core of "who can access what" — the authorization layer.

7.1 How principals work

A certificate contains a list of principals (e.g., `alice`, `deploy`, `backend-devs`). The server checks: "Is any of this certificate's principals allowed to log in as the requested system user?"

7.2 AuthorizedPrincipalsFile

On the server, create principal files per system user:

```
# /etc/ssh/sshd_config
AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u
```

`%u` expands to the target username. Create the mapping files:

```
# Who can log in as "deploy"
sudo mkdir -p /etc/ssh/auth_principals

echo -e "backend-devs\nsre-team\nalice" | sudo tee /etc/ssh/auth_principals/deploy
# Result: anyone with principal "backend-devs", "sre-team", or "alice" can SSH as
deploy

# Who can log in as "postgres"
echo -e "dba-team\nbob" | sudo tee /etc/ssh/auth_principals/postgres
# Result: only "dba-team" or "bob" principals can SSH as postgres

# Who can log in as "root" (keep this very restricted)
echo "sre-lead" | sudo tee /etc/ssh/auth_principals/root
```

7.3 Authorization matrix example

User	Group/Principal in Cert	Can SSH as `deploy`?	Can SSH as `postgres`?	Can SSH as `root`?
Alice	`alice`, `backend-devs`	Yes (both match)	No	No
Bob	`bob`, `dba-team`	No	Yes (both match)	No
Carol	`carol`, `sre-team`	Yes (`sre-team`)	No	No
Dave	`dave`, `sre-lead`, `sre-team`	Yes (`sre-team`)	No	Yes (`sre-lead`)

7.4 AuthorizedPrincipalsCommand (dynamic lookup)

For large teams, use a script instead of static files:

```
# /etc/ssh/sshd_config
AuthorizedPrincipalsCommand /usr/local/bin/check-principals.sh %u %i
AuthorizedPrincipalsCommandUser nobody
```

Example script (`/usr/local/bin/check-principals.sh`):

```
#!/bin/bash
# %u = target username, %i = certificate key ID
TARGET_USER="$1"
```

```
KEY_ID="$2"

# Query an API, LDAP, or database for allowed principals
curl -s "https://access-api.internal/principals?user=${TARGET_USER}" 2>/dev/null
```

This lets you manage access centrally without touching server files.

---

8. Part 6: Production CA Server with HashiCorp Vault

8.1 Why Vault?

- CA private key is stored in Vault's encrypted backend (or an HSM via Vault's seal)
- Signing requests go through Vault's auth + policy engine
- Full audit log of every cert issued
- Integrates with OIDC, LDAP, GitHub, etc. for authentication

8.2 Install and initialize Vault

```
# Install (Ubuntu/Debian)
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" \
  | sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install vault

# Start in dev mode for learning (NOT for production)
vault server -dev &
export VAULT_ADDR='http://127.0.0.1:8200'
```

8.3 Enable the SSH secrets engine

```
vault secrets enable -path=ssh-client-signer ssh
```

8.4 Generate or import a CA key

```
# Option A: Let Vault generate the CA key (recommended — key never leaves Vault)
vault write ssh-client-signer/config/ca generate_signing_key=true

# Option B: Import an existing CA key
vault write ssh-client-signer/config/ca \
  private_key=@ca/ca_key \
  public_key=@ca/ca_key.pub
```

Retrieve the public key to distribute to servers:

```
vault read -field=public_key ssh-client-signer/config/ca > ca_key.pub
```

8.5 Create signing roles (policies)

Each role defines what kind of certificates can be issued:

```
# Role for backend developers
vault write ssh-client-signer/roles/backend-dev - <<EOF
{
  "algorithm_signer": "rsa-sha2-256",
  "allow_user_certificates": true,
  "allowed_users": "deploy,app-user",
  "allowed_extensions": "permit-pty",
  "default_extensions": {
    "permit-pty": ""
  },
  "key_type": "ca",
  "default_user": "deploy",
  "ttl": "8h",
  "max_ttl": "24h"
}
EOF

# Role for DBAs
vault write ssh-client-signer/roles/dba - <<EOF
{
  "algorithm_signer": "rsa-sha2-256",
  "allow_user_certificates": true,
  "allowed_users": "postgres",
  "allowed_extensions": "permit-pty",
  "default_extensions": {
    "permit-pty": ""
  },
  "key_type": "ca",
  "default_user": "postgres",
  "ttl": "4h",
  "max_ttl": "8h"
}
EOF

# Role for CI/CD (very restricted)
vault write ssh-client-signer/roles/ci-deploy - <<EOF
{
  "algorithm_signer": "rsa-sha2-256",
  "allow_user_certificates": true,
  "allowed_users": "deploy",
  "allowed_extensions": "",
  "default_extensions": {
    "permit-pty": "",
    "force-command": "/usr/local/bin/deploy.sh"
  },
  "key_type": "ca",
  "ttl": "30m",
  "max_ttl": "1h"
```

```
}
EOF
```

8.6 Configure Vault authentication

```
# OIDC (Google, Okta, etc.)
vault auth enable oidc
vault write auth/oidc/config \
  oidc_discovery_url="https://accounts.google.com" \
  oidc_client_id="YOUR_CLIENT_ID" \
  oidc_client_secret="YOUR_SECRET" \
  default_role="default"

# Map OIDC groups to Vault policies
vault write auth/oidc/role/default \
  bound_audiences="YOUR_CLIENT_ID" \
  allowed_redirect_uris="http://localhost:8250/oidc/callback" \
  user_claim="email" \
  groups_claim="groups" \
  policies="ssh-default"
```

8.7 Create Vault policies

```
# Policy: backend devs can sign with the backend-dev role
vault policy write ssh-backend-dev - <<EOF
path "ssh-client-signer/sign/backend-dev" {
  capabilities = ["create", "update"]
}
EOF

# Policy: DBAs can sign with the dba role
vault policy write ssh-dba - <<EOF
path "ssh-client-signer/sign/dba" {
  capabilities = ["create", "update"]
}
EOF
```

8.8 Developer workflow — signing a key via Vault

```
# 1. Authenticate to Vault (one-time or when token expires)
vault login -method=oidc

# 2. Sign your public key
vault write -field=signed_key ssh-client-signer/sign/backend-dev \
  public_key=@~/.ssh/id_ed25519.pub > ~/.ssh/id_ed25519-cert.pub

# 3. Verify
ssh-keygen -L -f ~/.ssh/id_ed25519-cert.pub
```

```
# 4. Connect — cert is automatically used
ssh deploy@backend-server1.example.com
```

8.9 Enable audit logging

```
vault audit enable file file_path=/var/log/vault/audit.log
```

Every cert issuance is now logged with:

- Who requested it (identity from auth method)
- What role they used
- What principals were in the cert
- When it was issued and when it expires
- Source IP of the request

---

9. Part 7: Revocation

9.1 Key Revocation Lists (KRL)

For revoking certificates before they expire (compromised key, fired employee, etc.):

```
# Create a KRL revoking a specific certificate
ssh-keygen -k -f /etc/ssh/revoked_keys -s ca/ca_key.pub users/alice-cert.pub

# Update an existing KRL (add more revoked certs)
ssh-keygen -k -u -f /etc/ssh/revoked_keys -s ca/ca_key.pub users/bob-cert.pub
```

On the server, add to `/etc/ssh/sshd_config`:

```
RevokedKeys /etc/ssh/revoked_keys
```

9.2 KRL distribution

The challenge with KRLs is distributing them to all servers. Options:

- **Cron + rsync/scp** — simple, pull from central location
- **Configuration management** (Ansible, Chef, Puppet) — push KRL updates
- **Consul/etcd watch** — event-driven distribution

9.3 Short TTL as primary defense

The best revocation strategy is **not needing it**:

TTL	Use case	Revocation needed?
8 hours	Daily developer access	Rarely — cert expires by EOD
30 minutes	CI/CD pipeline	Almost never
5 minutes	Emergency break-glass	No

52 weeks	Host certificates	Yes — maintain KRL for hosts

---

10. Part 8: Automation and Tooling

10.1 Helper script for developers

Create a wrapper so devs don't need to remember Vault commands:

```
#!/bin/bash
# ssh-cert-login.sh — fetch a short-lived SSH certificate

set -euo pipefail

VAULT_ADDR="${VAULT_ADDR:-https://vault.internal:8200}"
ROLE="${1:-backend-dev}"
KEY="${2:-$HOME/.ssh/id_ed25519}"
CERT="${KEY}-cert.pub"

# Check if current cert is still valid
if [ -f "$CERT" ]; then
    EXPIRY=$(ssh-keygen -L -f "$CERT" 2>/dev/null | grep "Valid:" | grep -oP 'to
\K[^ ]+')
    if [ -n "$EXPIRY" ]; then
        EXPIRY_EPOCH=$(date -d "$EXPIRY" +%s 2>/dev/null || echo 0)
        NOW_EPOCH=$(date +%s)
        if [ "$NOW_EPOCH" -lt "$EXPIRY_EPOCH" ]; then
            echo "Current cert still valid until $EXPIRY"
            exit 0
        fi
    fi
fi

# Authenticate if needed
if ! vault token lookup &>/dev/null; then
    echo "Authenticating to Vault..."
    vault login -method=oidc
fi

# Sign the key
echo "Requesting certificate for role: $ROLE"
vault write -field=signed_key "ssh-client-signer/sign/${ROLE}" \
    public_key=@"${KEY}.pub" > "$CERT"

chmod 600 "$CERT"

echo "Certificate issued:"
ssh-keygen -L -f "$CERT" | grep -E "(Valid|Principals|Key ID)"
```

Usage:

```
# Default role
./ssh-cert-login.sh

# Specific role
./ssh-cert-login.sh dba
```

10.2 SSH config for automatic cert usage

Add to `~/.ssh/config`:

```
Host *.example.com
    IdentityFile ~/.ssh/id_ed25519
    CertificateFile ~/.ssh/id_ed25519-cert.pub
    # Optional: run the helper script before connecting
    # ProxyCommand bash -c '~/ssh-cert-login.sh backend-dev >/dev/null 2>&1; nc %h %p'
```

10.3 Ansible playbook for server setup

```yaml
# setup-ssh-ca.yml
---
- name: Configure SSH CA trust on servers
  hosts: all
  become: true
  vars:
    ca_public_key: "ssh-ed25519 AAAAC3Nza...your-ca-pubkey... SSH CA"

  tasks:
    - name: Install CA public key
      copy:
        content: "{{ ca_public_key }}"
        dest: /etc/ssh/ca_key.pub
        owner: root
        group: root
        mode: "0644"

    - name: Configure sshd to trust CA
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: "^#?TrustedUserCAKeys"
        line: "TrustedUserCAKeys /etc/ssh/ca_key.pub"
      notify: restart sshd

    - name: Configure AuthorizedPrincipalsFile
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: "^#?AuthorizedPrincipalsFile"
        line: "AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u"
      notify: restart sshd

    - name: Create auth_principals directory
```

```yaml
    file:
      path: /etc/ssh/auth_principals
      state: directory
      owner: root
      group: root
      mode: "0755"

  - name: Configure principals for deploy user
    copy:
      content: |
        backend-devs
        sre-team
      dest: /etc/ssh/auth_principals/deploy
      owner: root
      group: root
      mode: "0644"

  - name: Configure revoked keys file
    lineinfile:
      path: /etc/ssh/sshd_config
      regexp: "^#?RevokedKeys"
      line: "RevokedKeys /etc/ssh/revoked_keys"
    notify: restart sshd

  - name: Create empty revoked keys file if absent
    file:
      path: /etc/ssh/revoked_keys
      state: touch
      owner: root
      group: root
      mode: "0644"

handlers:
  - name: restart sshd
    systemd:
      name: sshd
      state: restarted
```

Run it:

```
ansible-playbook -i inventory.ini setup-ssh-ca.yml
```

---

11. Architecture Diagrams

Learning/Small Team Setup

```
┌──────────────────────────────────────────────────────────────┐
│                      CA Operator (you)                       │
│                                                              │
│  ca_key (private, passphrase-protected)                      │
```

```
│  ca_key.pub (distributed to servers)                        │
└──────────┬──────────────────────────────────────┬──────────┘
           │ signs user keys                      │ signs host keys
           ▼                                      ▼
┌──────────────────┐                  ┌──────────────────────────┐
│  Developer       │                  │  Server                  │
│                  │                  │                          │
│  id_ed25519      │    SSH with      │  TrustedUserCAKeys       │
│  id_ed25519      │──────certificate─│  /etc/ssh/ca_key.pub     │
│   -cert.pub      │                  │                          │
│                  │                  │  AuthorizedPrincipalsFile │
│                  │                  │  /etc/ssh/auth_principals/│
└──────────────────┘                  └──────────────────────────┘
```

Production Setup

```
┌──────────┐      ┌──────────┐      ┌──────────────┐      ┌──────────┐
│Developer │──────►│ Identity │──────►│  CA Server   │──────►│ Signing  │
│          │ SSO  │ Provider │Valid │  (Vault)     │Sign  │ Backend  │
│          │      │(Okta/OIDC)│Token│              │Req   │ (HSM)    │
└──────────┘      └──────────┘      └──────────────┘      └──────────┘
     │                                     │
     │ SSH with cert                       │ Audit log
     ▼                                     ▼
┌──────────┐                        ┌──────────────┐
│  Server  │                        │  SIEM /      │
│          │                        │  Log Store   │
│ ca_key.pub (trusts CA)            │              │
│ auth_principals/ (access rules)   └──────────────┘
│ revoked_keys (KRL)                │
└──────────┘
```

Certificate Signing Flow

```
  Developer                    CA Server                    Server
     │                            │                            │
     │  1. Generate key pair      │                            │
     │  (id_ed25519 + .pub)       │                            │
     │                            │                            │
     │  2. Authenticate (SSO)     │                            │
     │ ──────────────────────────►│                            │
     │                            │                            │
     │  3. Send public key +      │                            │
     │     auth token             │                            │
     │ ──────────────────────────►│                            │
     │                            │                            │
     │                  4. Validate token                      │
     │                  5. Check policy                        │
     │                  6. Sign key with CA                    │
     │                            │                            │
     │  7. Receive signed cert    │                            │
```

```
    | ←──────────────────    |                          |
    |                        |                          |
    |  8. SSH connect with cert                         |
    | ──────────────────────────────────────────→       |
    |                                                   |
    |                           9. Verify cert against CA pub|
    |                          10. Check principals match    |
    |                          11. Check cert not expired    |
    |                          12. Check KRL (not revoked)   |
    |                                                   |
    |  13. Access granted                               |
    | ←──────────────────────────────────────────       |
```

---

12. Quick Reference

Commands cheat sheet

```
# === CA SETUP ===
# Generate CA key pair
ssh-keygen -t ed25519 -f ca_key -C "SSH CA"

# === USER CERTIFICATES ===
# Sign a user key (8h, specific principals)
ssh-keygen -s ca_key -I "user-id" -n principal1,principal2 -V +8h user.pub

# Sign with restrictions
ssh-keygen -s ca_key -I "user-id" -n deploy -V +1h \
  -O no-port-forwarding -O source-address=10.0.0.0/8 user.pub

# === HOST CERTIFICATES ===
# Sign a host key
ssh-keygen -s ca_key -I "host-id" -h -n hostname,ip -V +52w host_key.pub

# === INSPECTION ===
# View certificate details
ssh-keygen -L -f certificate-cert.pub

# === REVOCATION ===
# Create KRL
ssh-keygen -k -f revoked_keys -s ca_key.pub cert-to-revoke-cert.pub

# Update KRL
ssh-keygen -k -u -f revoked_keys -s ca_key.pub another-cert.pub

# === VAULT ===
# Sign key via Vault
vault write -field=signed_key ssh-client-signer/sign/role-name \
  public_key=@~/.ssh/id_ed25519.pub > ~/.ssh/id_ed25519-cert.pub
```

Server sshd_config template

```
# /etc/ssh/sshd_config — CA-related lines

# Trust user certs signed by this CA
TrustedUserCAKeys /etc/ssh/ca_key.pub

# Map principals to system users
AuthorizedPrincipalsFile /etc/ssh/auth_principals/%u

# Host certificate (server proves its identity)
HostCertificate /etc/ssh/ssh_host_ed25519_key-cert.pub

# Revocation list
RevokedKeys /etc/ssh/revoked_keys

# Optional: disable plain key auth once CA is working
# AuthorizedKeysFile none
```

---

13. Troubleshooting

Common issues

"Permission denied (publickey)"

```
# 1. Check cert is valid
ssh-keygen -L -f ~/.ssh/id_ed25519-cert.pub
# Look at Valid: line — is it expired?

# 2. Check principals match
# The cert's principals must appear in /etc/ssh/auth_principals/<target-user>

# 3. Verbose SSH to see what's happening
ssh -vvv -i ~/.ssh/id_ed25519 deploy@server 2>&1 | grep -i cert
```

"Certificate invalid: name is not a listed principal"

The username you're SSHing as isn't in the cert's principals list.

```
# Check what principals the cert has
ssh-keygen -L -f ~/.ssh/id_ed25519-cert.pub | grep Principals -A 10

# Check what principals the server allows for the target user
cat /etc/ssh/auth_principals/<target-user>
```

"Certificate invalid: expired"

```
# Get a new cert — the old one's TTL ran out
vault write -field=signed_key ssh-client-signer/sign/role \
```

```
  public_key=@~/.ssh/id_ed25519.pub > ~/.ssh/id_ed25519-cert.pub
```

Host certificate not working

```
# Verify the known_hosts entry
# Must be: @cert-authority *.example.com <ca_key.pub contents>
grep cert-authority ~/.ssh/known_hosts

# Check the host cert on the server
ssh-keygen -L -f /etc/ssh/ssh_host_ed25519_key-cert.pub

# Ensure the hostname you're connecting to matches a principal in the host cert
```

sshd won't start after config change

```
# Test config before restarting
sudo sshd -t

# Check for syntax errors
sudo sshd -T | grep -i error
```

Next Steps

1. **Start with Part 1-3** locally to understand signing
2. **Set up a test server** (VM or container) for Parts 4-5
3. **Deploy Vault** for production (Part 6)
4. **Automate** server provisioning with Ansible (Part 8)
5. **Disable plain key auth** once CA is fully rolled out (`AuthorizedKeysFile none`)